SIC Lab
Programmatic Bluetooth in C

version 1.0

Authors: André Zúquete, Vitor Cunha

Version log:

- 1.0: Initial version

1 Introduction

Bluetooth is a wireless technology originally designed to replace cables. The first version was strongly associated with replacing RS-232 serial connections, and Bluetooth has since evolved into a general-purpose way of connecting personal devices such as earbuds, smartwatches, sensors, smartplugs, and other peripherals.

In the previous practical lecture we introduced and explored BlueZ, the Bluetooth stack used by Linux. Linux applications can interact with BlueZ through management utilities such as `bluetoothctl` or programmatically through D-Bus.

In this practical we use our own C programs to perform several Bluetooth tasks. The supplied project uses GLib-2.0 and Gio-2.0, in particular `GDBusConnection`, `GDBusProxy`, `GVariant`, and the GLib main loop.

The supplied project builds five applications:

- `scan`: discover nearby Bluetooth devices for 20 seconds and display basic device state.
- `scan-plus`: discover devices and continue receiving D-Bus property-change notifications while it is running.
- `scan-services`: discover devices, connect to each discovered device asynchronously, and display its Bluetooth service UUIDs as service information becomes available.
- `name`: read or change the local Bluetooth adapter `Alias`.
- `just_works`: register a BlueZ Agent1 object implementing a no-input/no-output pairing policy, make the adapter pairable and discoverable, and wait for pairing requests.

The practical therefore covers discovery, D-Bus event handling, asynchronous connection and service enumeration, adapter properties, and a pairing agent. It does not contain a Bluetooth chat application.

2 Setup

Install the compiler, build tools, and GLib/GIO development files on Debian:

```
$ sudo apt install gcc make libglib2.0-dev
```

The project is built with `pkg-config` through the supplied `Makefile`. The Makefile obtains compilation flags from `glib-2.0` and link flags from `gio-2.0`.

Unpack the supplied ZIP file and enter the directory containing the C sources and `Makefile`. Then build the applications:

```
$ make
```

The expected applications are:

```
scan - Discover Devices
scan-plus
scan-services
name
just_works
```

To remove the generated object files and executables:

```
$ make clean
```

The source tree is organised around reusable modules:

- `dbus.c` / `dbus.h`: obtain a connection to the system D-Bus.
- `exit.c` / `exit.h`: release resources and terminate after a fatal D-Bus error.
- `subscribe.c` / `subscribe.h`: subscribe to `InterfacesAdded` and `PropertiesChanged` D-Bus signals.
- `filter.c` / `filter.h`: install a BlueZ discovery filter.
- `discovery.c` / `discovery.h`: start and stop discovery on `hci0`.
- `connect.c` / `connect.h`: provide a synchronous helper for the BlueZ `Device1.Connect` method.
- `gatt.c` / `gatt.h`: inspect BlueZ managed objects and list device service UUIDs.
- `set_adapter_property.c` / `set_adapter_property.h`: set boolean properties of the local BlueZ adapter through D-Bus.
- `just_works_agent.c` / `just_works_agent.h`: register the `org.bluez.Agent1` object used by `just_works`.
- `scan.c`, `scan-plus.c`, `scan-services.c`: discovery and service examples.
- `name.c`: read or modify the adapter `Alias` property.
- `just_works.c`: register the pairing agent, make the adapter pairable and discoverable, and enter the GLib main loop.
- `Makefile`: compile and link all five applications.

The archive also contains `subscribe_new_interfaces.h`, which declares the same subscription functions present in `subscribe.h`. The supplied source files use `subscribe.h`.

3 Working with the BlueZ D-Bus API in C

The examples in this project use the Linux system D-Bus. BlueZ exposes Bluetooth adapters and remote devices as D-Bus objects. The programs issue method calls to BlueZ and subscribe to D-Bus signals so that device discovery and property changes can be processed asynchronously.

A useful way to read the project is to follow this path:

```
D-Bus connection
      |
      +--> discovery filter
      |
      +--> StartDiscovery
      |
      +--> InterfacesAdded signals
      |
      +--> device properties
      |       |
      |       +--> PropertiesChanged signals
      |       |
      |       +--> Connect
      |       |
      |       +--> service UUIDs
      |
      +--> adapter properties
      |
      +--> Agent1 method calls during pairing
```

The exercises below follow the same progression.

4 Scanning for devices

4.1 What the program does

Start with `scan.c`.

The program obtains a system-bus connection with `get_dbus_handle()`, subscribes to BlueZ `InterfacesAdded` signals, installs a discovery filter, and calls `StartDiscovery` on `/org/bluez/hci0`.

The discovery filter is created by `install_filter()` in `filter.c`. The `scan` application passes the transport value `"le"`, so the program requests BLE discovery. The filter also sets `DuplicateData` to `TRUE` in the D-Bus request. The program prints the transport selected by the filter and reports that the "no duplicate data" filter was installed.

Each newly added `org.bluez.Device1` object is handled by `on_interfaces_added()`. The callback looks for these properties when they are present:

- `Address`
- `Name`
- `Paired`
- `Bonded`
- `Connected`
- `Trusted`

When an address is available, the application prints the four boolean state values followed by the address and, when available, the device name.

The program uses a GLib main loop to receive asynchronous D-Bus signals. A `SIGALRM` handler stops the main loop after 20 seconds. Once the loop ends, `scan` calls `StopDiscovery()` and releases the D-Bus resources.

4.2 Run the program

```
$ ./scan
```

Let it run for the complete scan period. Then answer:

1. How many Bluetooth devices did you find? 84 (?)
2. Can you identify your own Bluetooth devices in the output?
3. Which reported values identify a device and which values describe its state? ultimos values seão MAC addr e (se tiver) nome
4. Does repeating the scan produce exactly the same list?
não, depende dos dispositivos ativos no momento

4.3 Exercise: understand the discovery filter

Open `scan.c` and change:

```
result = install_filter( bus, "le", &error );
```

to use the transport value `"auto"`.

Rebuild and run `scan` again:

```
$ make
$ ./scan
```

Compare the results with the original BLE-only scan.

Questions:

1. Does the number of detected devices change? é suposto serem mais pq fazemos o dan scan por devices LE e classic

2. Do devices that were absent from the BLE-only scan now appear? sim
3. Which function actually sends the `SetDiscoveryFilter` request to BlueZ?
4. Where is the transport value passed from the application into the filter?

4.4 Exercise: the 20-second timeout

The timeout is hard-coded in `scan.c`:

```
alarm( 20 );
```

Modify the application so that the timeout can be supplied on the command line. For example:

```
$ ./scan 5
$ ./scan 30
```

Parse the argument and replace the fixed value passed to `alarm()`.

Test several values and record:

1. Whether the number of devices found changes. → não
2. Whether longer scans produce additional devices. → sim
3. How the scan duration affects the practical usability of the tool.

5 Observing device property changes

Now inspect `scan-plus.c`.

This program keeps the discovery mechanism but adds a second D-Bus subscription:

```
subscribe_device_properties_changed( bus, NULL, on_properties_changed, NULL );
```

The `NULL` device path means that the subscription applies to all object paths matched by the signal subscription rather than one particular device. The callback only processes changes belonging to the `org.bluez.Device1` interface.

The callback receives the `PropertiesChanged` parameters as:

```
(&s@a{sv}@as)
```

and iterates over the changed-property dictionary. Every changed value is formatted with `g_variant_print()` before being displayed.

The callback also derives the Bluetooth address from the end of the object path. The source assumes the path has the BlueZ device form ending in a 17-character address such as `xx_xx_xx_xx_xx_xx`.

Unlike `scan`, this program does not install a 20-second alarm. Its GLib main loop keeps running until the process is stopped, for example with `Ctrl+C`.

5.1 Run the program

```
$ ./scan-plus
```

Then interact with nearby Bluetooth devices or move a device in and out of range and observe the property changes reported by BlueZ.

5.2 Exercise: compare `scan` and `scan-plus`

Identify the additional information made visible by the `PropertiesChanged` subscription.

1. Which properties appear repeatedly as the environment changes?
2. Which values are strings, booleans, or other D-Bus variant types?
3. Why does `scan-plus` need a GLib main loop? → Porque DBus signals são assincronos
4. What new signal subscription is added compared with `scan`?

5.3 Exercise: inspect the callback path

Trace this sequence in the source code:

```
subscribe_device_properties_changed()
        -> PropertiesChanged signal
        -> on_properties_changed()
        -> g_variant_get()
        -> iterate over changed properties
        -> g_variant_print()
        -> print each property
```

Write down the D-Bus interface and signal name involved at each step.

5.4 Exercise: inspect assumptions made by the callback

The callback assumes that the object path ends with the Bluetooth address and computes the address by taking the last 17 characters. What would happen if a signal for an object with an unexpected path reached this callback?

Discuss how you would make this code more robust before using it in a general D-Bus monitoring application.

6 Connecting to discovered devices and listing services

The third application combines discovery with asynchronous connection and service enumeration. Read `scan-services.c`, `connect.c`, and `gatt.c` together.

6.1 The connection helper

`connect.c` provides:

```
GVariant * connect_device( GDBusConnection *bus,
                           const char *device,
                           int timeout,
                           GError **error );
```

This helper performs a synchronous D-Bus call to the BlueZ `org.bluez.Device1` interface and invokes the `Connect` method.

`scan-services.c` deliberately does not use this synchronous helper for its connection operation. Instead, it creates a `GDBusProxy` for `org.bluez.Device1` and calls `Connect` asynchronously with `g_dbus_proxy_call()`. The example uses a 30,000 millisecond timeout.

This distinction is important: the device-discovery callback should not block while waiting for a potentially slow Bluetooth connection to finish.

6.2 Listing service UUIDs

`gatt.c` provides `list_gatt_services()`.

The function calls the D-Bus `org.freedesktop.DBus.ObjectManager` method `GetManagedObjects` on `/`. It iterates over the returned object paths and keeps only paths below the discovered device path.

For each matching object, the function looks for the `org.bluez.Device1` interface and then the `UUIDs` property. Every UUID is printed. If the `UUIDs` property is not available, it prints `No services` for that object.

6.3 Waiting for service resolution

`scan-services.c` subscribes to `PropertiesChanged` notifications for the specific device after the asynchronous connection completes.

The callback reads the `ServicesResolved` property through another D-Bus `org.freedesktop.DBus.Properties.Get` call.

Two situations can therefore be observed:

1. `ServicesResolved` is false. The program reports that services are not yet resolved and attempts another service listing.
2. `ServicesResolved` is true. The program reports that the services are resolved, lists them, and removes the subscription for that device.

This demonstrates an important asynchronous distinction: connecting to a Bluetooth device and having all service information represented in BlueZ are separate events.

6.4 Run the application

```
$ ./scan-services
```

The program uses the discovery filter `"auto"`, so it does not restrict the transport to BLE in the way `scan` does.

For each discovered device, inspect the output and identify:

- its Bluetooth address;
- its optional name;
- the reported `Paired`, `Bonded`, `Connected`, and `Trusted` state;
- the fact that a connection is requested automatically;
- the initial service listing after the connection callback;
- subsequent service listings while `ServicesResolved` is false;
- the final service listing when `ServicesResolved` becomes true.

6.5 Exercise: follow one device through the program

Choose one discovered device and trace it through the source:

```
InterfacesAdded
      |
      v
create GDBusProxy for org.bluez.Device1
      |
      v
asynchronous Connect()
      |
      v
on_connect_done()
      |
      +--> initial list_gatt_services()
      |
      v
subscribe to PropertiesChanged for this device
      |
      v
read ServicesResolved
      |
      +--> false -> list services again
      |
      +--> true  -> list services and unsubscribe
```

Locate the corresponding function in `scan-services.c` for every arrow.

6.6 Exercise: why is the connection asynchronous?

Compare the connection logic in `scan-services.c` with the synchronous helper in `connect.c`.

Explain why performing the connection directly and synchronously from an `InterfacesAdded` callback could prevent the program from processing other D-Bus events while it waits.

6.7 Exercise: service discovery timing

Run `scan-services` several times and observe whether the service UUIDs are already available at the first listing or only become complete after one or more `PropertiesChanged` notifications.

Record one example and explain it using the `ServicesResolved` property.

6.8 Exercise: automatic connection policy

The source connects to every discovered device that reaches the device callback. There is no user-selection step.

Modify the code so that connection only happens when a condition of your choice is met, for example a selected address or device name.

Explain where in `on_interfaces_added()` the policy check should be inserted and why it should happen before creating the asynchronous connection.

7 Changing the local Bluetooth adapter name

A Bluetooth adapter exposes the `Name` and `Alias` properties through `org.bluez.Adapter1`. This project uses `Alias` as the user-visible name.

The program in `name.c` can either read the current `Alias` or set a new value. The modification is made immediately on the local adapter; other hosts will only observe the change after they obtain updated information from the adapter.

`name.c` accepts zero or one argument. More than one argument is rejected with a usage message.

7.1 Read the current adapter name

```
$ ./name
```

The program calls the D-Bus `Get` method for:

```
org.bluez.Adapter1
property: Alias
```

It prints:

```
Bluetooth adapter name: <name>
```

7.2 Set the adapter name

```
$ ./name new_name
```

The program calls the D-Bus `Set` method with the new string and reports:

```
Bluetooth adapter name changed to: new_name
```

7.3 Exercise: inspect property access

Identify the differences between the `Get` and `Set` D-Bus calls in `name.c`.

1. What interface and methods are used?
2. How is the `Alias` property identified?
3. Why does the `Get` call expect a result of type `(v)`?
4. Why does the `Set` call use `(ssv)` parameters?

7.4 Exercise: observe the change remotely

Set a new adapter name and then have another Bluetooth-capable host scan for the adapter. Determine when the new name becomes visible remotely.

8 Implementing a JustWorks pairing agent

The project also includes a pairing agent. This is implemented by `just_works.c` and `just_works_agent.c`.

A BlueZ Agent is a D-Bus object implementing `org.bluez.Agent1`. BlueZ calls the agent when a pairing operation needs a policy decision or user interaction.

The example creates the agent object at:

```
/sic/example/justworks_agent
```

and registers it with `org.bluez.AgentManager1` using the capability:

```
NoInputNoOutput
```

It then requests that this agent become the default agent.

8.1 What `just_works.c` does

The application performs these operations in order:

1. Obtain the system D-Bus connection.
2. Register the local `org.bluez.Agent1` object implemented by `just_works_agent.c`.
3. Call `RegisterAgent` on `/org/bluez` using `NoInputNoOutput`.
4. Call `RequestDefaultAgent` so this agent becomes the default BlueZ agent.
5. Set the local adapter `Pairable` property to `TRUE`.
6. Set the local adapter `Discoverable` property to `TRUE`.
7. Enter a GLib main loop and wait for BlueZ to invoke Agent1 methods.

The application does not start a finite scan and has no termination timer. Stop it externally, for example with `Ctrl+C`.

8.2 What `just_works_agent.c` implements

The introspection XML defines the standard Agent1 methods used by the example:

- `Release`
- `RequestPinCode`
- `DisplayPinCode`
- `RequestPasskey`

- `DisplayPasskey`
- `RequestConfirmation`
- `RequestAuthorization`
- `AuthorizeService`
- `Cancel`

The callback `handle_agent_method_call()` applies the following policy:

- `RequestAuthorization` is accepted automatically.
- `AuthorizeService` is accepted automatically.
- `RequestConfirmation` is automatically accepted and the received passkey is printed.
- `DisplayPinCode` and `DisplayPasskey` are accepted and their values are printed.
- `RequestPinCode` and `RequestPasskey` are rejected because the agent has `NoInputNoOutput` capability and therefore cannot provide a PIN or passkey.
- `Release` and `Cancel` are acknowledged.
- An unknown method is rejected with `org.bluez.Error.Rejected`.

This is therefore a simple no-user-input pairing policy. In particular, the implementation also accepts a `RequestConfirmation` callback instead of rejecting it.

8.3 Run the application

```
$ ./just_works
```

The application should remain running. When another device attempts a pairing operation that reaches the registered agent, observe the messages printed by `handle_agent_method_call()`.

8.4 Exercise: inspect the Agent1 object

Start from `just_works_agent.c` and identify these four elements:

```
introspection XML
       |
       v
handle_agent_method_call()
       |
       v
agent_vtable
       |
       v
g_dbus_connection_register_object()
```

Explain how the XML description and the method-call callback together expose a D-Bus object that BlueZ can call.

8.5 Exercise: analyse the pairing policy

For each Agent1 method listed above, record whether the example:

- accepts the request;
- rejects the request; or
- simply acknowledges the method while printing information.

Then answer:

1. Which operations can proceed without human approval?
2. Which pairing mechanisms cannot be completed because the agent has no input capability?
3. Why is automatically accepting `RequestConfirmation` different from a strict JustWorks-only policy?

8.6 Exercise: change the policy

Modify one branch of `handle_agent_method_call()` so that a selected operation is rejected instead of automatically accepted.

Test the modified policy and explain the D-Bus error returned to BlueZ.

9 Understanding the reusable D-Bus modules

After running the applications, return to the common source files.

9.1 `dbus.c`

`get_dbus_handle()` uses `g_bus_get_sync()` with `G_BUS_TYPE_SYSTEM`. BlueZ is therefore accessed through the system bus.

9.2 `filter.c`

`install_filter()` builds a D-Bus dictionary (`a{sv}`) containing:

- `Transport`, whose value is supplied by the caller (`"le"` in `scan`, `"auto"` in `scan-plus` and `scan-services`);
- `DuplicateData`, set to `TRUE`.

The resulting dictionary is passed to BlueZ's `Adapter1.SetDiscoveryFilter`.

9.3 `discovery.c`

`start_discovery()` and `stop_discovery()` are wrappers around the BlueZ `org.bluez.Adapter1` methods `StartDiscovery` and `StopDiscovery` on `/org/bluez/hci0`.

Both calls are synchronous D-Bus calls with no explicit timeout.

9.4 `subscribe.c`

The project uses `g_dbus_connection_signal_subscribe()` for two purposes:

- `org.freedesktop.DBus.ObjectManager.InterfacesAdded`, which reports newly added BlueZ objects;
- `org.freedesktop.DBus.Properties.PropertiesChanged`, which reports changes to object properties.

The second subscription can optionally be restricted to one object path, which is used by `scan-services` after connecting to a specific device.

9.5 `connect.c`

`connect_device()` is a synchronous convenience function around `org.bluez.Device1.Connect`. The current `scan-services` application keeps it in the project but instead uses `GDBusProxy` and `g_dbus_proxy_call()` to perform its connection asynchronously.

9.6 `gatt.c`

`list_gatt_services()` obtains BlueZ's complete managed-object tree using `org.freedesktop.DBus.ObjectManager.GetManagedObjects`. It filters paths below the selected device and reads the `UUIDs` property from `org.bluez.Device1`.

9.7 `set_adapter_property.c`

`set_adapter_property_boolean()` wraps the D-Bus `org.freedesktop.DBus.Properties.Set` method for boolean `org.bluez.Adapter1` properties. `just_works.c` uses it to set `Pairable` and `Discoverable`.

9.8 `just_works_agent.c`

The file demonstrates a complete D-Bus object implemented in C:

1. Define the interface in introspection XML.
2. Implement the method-call callback.
3. Create a `GDBusInterfaceVTable`.
4. Register the object with `g_dbus_connection_register_object()`.

10 Practical security discussion

The code is useful for discussing what information and actions become exposed when an application talks directly to the local Bluetooth stack.

10.1 Discovery and device metadata

The scanning programs obtain device addresses and, when available, names and state properties such as `Paired`, `Bonded`, `Connected`, and `Trusted`.

Discuss:

1. What information about nearby devices is revealed by discovery?
2. Which values identify a device and which describe its current state?
3. What privacy implications follow from collecting and storing such data?

10.2 Event-driven interfaces

The programs trust data arriving through D-Bus signals and parse it as `GVariant` values.

Discuss:

1. Why does the program check interfaces and expected value types?
2. What assumptions do the callbacks make about the presence and shape of data?
3. Which assumptions would need additional validation in a production program?

10.3 Automatic connections

`scan-services` automatically attempts to connect to every discovered device that reaches its callback path.

Discuss:

1. Why can automatic connection be an unexpected policy decision?
2. How would you modify the program so that a user must explicitly approve a device before connection?
3. Which address or property could be used to make that decision?

10.4 Automatic pairing decisions

`just_works` installs an agent that accepts several pairing-related requests without user interaction and also makes the adapter pairable and discoverable.

Discuss:

1. What changes in the security posture when pairing decisions are automated?
2. Why does the capability `NoInputNoOutput` limit which requests can be completed?
3. Why should automatic authorization and service approval be treated as a policy choice rather than merely a programming detail?

10.5 Adapter discoverability

`just_works` explicitly sets the local adapter's `Discoverable` and `Pairable` properties to `TRUE`.

Discuss the difference between:

- discovering remote devices;
- making the local adapter discoverable;
- allowing the local adapter to be paired.

10.6 Service enumeration

The service-listing code queries BlueZ's object model and prints service UUIDs. Discuss what an application can learn from service enumeration before it ever uses an application-level service.

11 Additional programming challenges

11.1 Challenge 1: parameterise the adapter

The project hard-codes `/org/bluez/hci0` in discovery, filtering, adapter properties, and some other D-Bus paths. Investigate how the programs would have to change to select another adapter path instead of always using `hci0`.

11.2 Challenge 2: improve the discovery output

Modify `scan.c` so that each device is printed in a more readable table or structured format. Keep the information extracted from `org.bluez.Device1`.

11.3 Challenge 3: add a safe connection policy

Modify `scan-services.c` so that it only connects to an explicitly selected device. Use a command-line address or another mechanism of your choice.

11.4 Challenge 4: make service matching explicit

`gatt.c` currently prints every UUID it finds. Modify it so that it can test for one particular UUID and report whether that service is present.

11.5 Challenge 5: use `connect_device()`

Create a small program that uses the synchronous helper in `connect.c` instead of creating a `GDBusProxy`. Compare its behaviour with the asynchronous connection used by `scan-services`.

11.6 Challenge 6: inspect the adapter property helper

Extend `set_adapter_property.c` so that it can set more than boolean values, or write a separate helper for string properties and compare it with the `name.c` implementation.

11.7 Challenge 7: improve the pairing policy

Change `just_works_agent.c` so that one selected device is accepted automatically while other devices are rejected. Explain where the device path enters the decision and how the policy differs from the current global behaviour.

11.8 Challenge 8: replace JustWorks with PIN-code pairing

Replace the current no-input/no-output pairing policy with a PIN-code-based pairing method. The supplied `just_works_agent.c` currently advertises the `NoInputNoOutput` capability and rejects `RequestPinCode` and `RequestPasskey` requests. Modify the agent so that it can participate in a PIN-code pairing exchange.

A possible implementation strategy is:

1. Change the capability registered by `just_works.c` to one appropriate for an agent that can accept PIN input.
2. Implement `RequestPinCode` in `handle_agent_method_call()` instead of rejecting it. Return a fixed PIN for the first version of the exercise.
3. Test the pairing process with a second Bluetooth device and determine where the PIN must be entered.
4. Replace the fixed PIN with a value supplied interactively or through a command-line argument.
5. Consider how the design changes if the PIN is generated dynamically rather than being a constant embedded in the source code.

Questions:

1. Which Agent1 method is invoked during PIN-code pairing?
2. Why is the `NoInputNoOutput` capability no longer appropriate?
3. Where should the PIN be validated or generated, and what would be the risks of storing a permanent PIN in the program?
4. How does this pairing flow differ from the current automatic acceptance of `RequestConfirmation` in the supplied agent?

11.9 Challenge 9: inspect error handling and assumptions

Review the source for places where a value is used without checking whether the corresponding D-Bus lookup succeeded. In particular, inspect the boolean device properties in `scan.c` and `scan-plus.c`, the device-path processing in `scan-plus.c`, and the D-Bus replies in the helper functions.

For each case, propose a small robustness improvement without changing the observable behaviour of the successful path.

11.10 Challenge 10: inspect the Makefile

The supplied `Makefile` builds five executables. Verify that:

- all required object files are linked for each target;
- `CFLAGS` is used when compiling `.c` files;
- `LIBS` supplies the GIO/GLib link dependencies;
- `make clean` removes every generated executable and object file.

12 Acknowledgements

Authored by André Zúquete and Vítor Cunha

13 Bibliography

- BlueZ
- GLib
- GIO / GDBus
- D-Bus